

Git : Contrôle de Version, Collaboration et Rollback

1. Introduction

- Git : système de **contrôle de version distribué (DVCS)**.
 - Permet de gérer l'historique des fichiers, collaborer et revenir à n'importe quelle version.
 - Utilisé par développeurs, équipes DevOps et entreprises pour le travail collaboratif.
-

2. Concepts Clés

Concept	Description
Repository (repo)	Projet + historique des modifications
Commit	Enregistrement d'un état du projet
Branch	Ligne de développement parallèle
Merge	Fusion de branches
Clone	Copier un dépôt distant localement
Pull / Push	Récupérer / envoyer les changements vers un dépôt distant
Staging	Zone intermédiaire avant le commit

3. Commandes Git Essentielles

3.1 Initialisation et configuration

```
git init                                # Créer un dépôt local

git clone <url>                          # Cloner un dépôt distant

git config --global user.name "Nom"

git config --global user.email "email@example.com"

git status                              # Voir l'état des fichiers

git remote -v                           # Lister dépôts distants
```

3.2 Gestion des fichiers

```
git add <fichier>                       # Ajouter au staging

git commit -m "message"                 # Valider les changements

git rm <fichier>                        # Supprimer un fichier

git mv <ancien> <nouveau>              # Renommer un fichier

git reset <fichier>                     # Retirer du staging
```

3.3 Branches et fusion

```
git branch                              # Lister les branches

git branch <nom>                        # Créer une branche

git checkout <branche>                  # Changer de branche

git checkout -b <branche>               # Créer + changer

git merge <branche>                     # Fusionner une branche

git rebase <branche>                    # Rejouer les commits

git branch -d <branche>                 # Supprimer une branche
```

3.4 Historique et comparaisons

<code>git log</code>	<code># Historique des commits</code>
<code>git log --oneline --graph</code>	<code># Historique simplifié</code>
<code>git diff</code>	<code># Modifications non commitées</code>
<code>git show <commit_id></code>	<code># Voir un commit précis</code>
<code>git blame <fichier></code>	<code># Qui a modifié chaque ligne</code>

3.5 Dépôts distants

<code>git remote add origin <url></code>	<code># Ajouter un dépôt distant</code>
<code>git fetch</code>	<code># Récupérer les branches distantes</code>
<code>git pull origin main</code>	<code># Récupérer + fusionner</code>
<code>git push origin main</code>	<code># Envoyer les commits locaux</code>

4. Collaboration – Bonnes pratiques

- Toujours `git pull` avant de travailler
- Créer une branche par fonctionnalité
- Commits fréquents avec messages clairs
- Pull Request / Code Review avant fusion
- Supprimer les branches fusionnées

Workflow recommandé :

```
git checkout -b feature1
git add .
git commit -m "Ajout feature1"
git push origin feature1
```

```
git checkout main
```

```
git pull origin main
```

```
git merge feature1
```

```
git push origin main
```

```
git branch -d feature1
```

```
git push origin --delete feature1
```

5. Rollback / Annuler des modifications

Situation	Commande	Exemple pratique
Annuler un fichier non commité	<code>git restore <fichier></code> ou <code>git checkout --</code> <code><fichier></code>	<code>git restore index.html</code>
Annuler toutes modifications locales	<code>git reset --hard</code>	<code>git reset --hard</code>
Annuler dernier commit (garder changements)	<code>git reset --soft HEAD~1</code>	<code>git reset --soft HEAD~1</code>
Annuler dernier commit et modifications	<code>git reset --hard HEAD~1</code>	<code>git reset --hard HEAD~1</code>
Annuler commit spécifique	<code>git revert <commit_id></code>	<code>git revert abc1234</code>
Rollback d'une branche complète	<code>git reset --hard</code> <code><commit_id> && git push</code> <code>--force</code>	<code>git reset --hard 9fceb02</code> <code>&& git push --force</code> <code>origin main</code>

Annuler fichier ajouté au staging

```
git reset <fichier>
```

```
git reset app.js
```

💡 Conseils :

- Préférer `git revert` sur dépôts partagés
- Toujours vérifier `git status` et `git log`
- Créer une branche avant un `reset --hard`

6. Git Tag – Gestion des versions

6.1 Liste des tags

```
git tag
```

6.2 Créer un tag léger

```
git tag v1.2
```

6.3 Créer un tag annoté

```
git tag -a v2.0 -m "Release version 2.0"
```

6.4 Taguer un commit spécifique

```
git tag -a v1.5 abc1234 -m "Tag version 1.5"
```

6.5 Voir les informations d'un tag

```
git show v2.0
```

6.6 Partager les tags

```
git push origin v2.0
```

```
git push origin --tags      # Tous les tags
```

6.7 Supprimer un tag

```
git tag -d v1.0
```

```
git push origin --delete v1.0
```

💡 Conseils pratiques :

- Utiliser les tags annotés (-a) pour les releases officielles
- Toujours pousser les tags après création pour collaboration
- Les tags servent pour **releases, rollback et déploiements**